Introduction to Bioinformatics

Automation & reusability

Matej Hrnčiar

05. 05. 2026

Bioinformatic pipelines are

Multi-step, file-dependent, and often computationally expensive ensembles of bioinformatic tools assembled together with a specific goal in mind

- Often re-run with small changes
- Automation?
- Reusability?

What can we use?

- Interns and PhD students :)
- Bash
- Makefile
- Python, R, Java, ...
- **Workflow managers**: Snakemake, Nextflow, AWS Genomics, Azure Genomics, Illumina DRAGEN

Automation

Bash, Make, Python

Why automate?

- So we don’t have to process samples manually
- Why should we keep track of all dependencies?
- Restart failed samples (in case of OOM or insufficient disk space errors)
- Use full capacity of a compute cluster

How can we skip processing of a file that already exists?

```
#!/bin/bash
qc_tool reads.fastq > reads.qc.txt

trim_tool reads.fastq > reads.trimmed.fastq

align_tool reference.fa reads.trimmed.fastq > reads.sam

sort_tool reads.sam > reads.sorted.bam

dedup_tool reads.sorted.bam > reads.sorted.dedup.bam
```

```bash
#!/bin/bash

if [ ! -f reads.qc.txt ]; then
    qc_tool reads.fastq > reads.qc.txt
fi

if [ ! -f reads.trimmed.fastq ]; then
    trim_tool reads.fastq > reads.trimmed.fastq
fi

if [ ! -f reads.sam ]; then
    align_tool reference.fa reads.trimmed.fastq > reads.sam
fi

if [ ! -f reads.sorted.bam ]; then
    sort_tool reads.sam > reads.sorted.bam
fi

if [ ! -f reads.sorted.dedup.bam ]; then
    dedup_tool reads.sorted.bam > reads.sorted.dedup.bam
fi
```

What if some tools can be run concurrently, can we run them in parallel?

Maybe Bash is not the best option for us… how about Makefile?

```bash
#!/bin/bash

qc_tool reads.fastq > reads.qc.txt &

trim_tool reads.fastq > reads.trimmed.fastq &

wait  # wait for QC + trimming to finish

align_tool reference.fa reads.trimmed.fastq > reads.sam

sort_tool reads.sam > reads.sorted.bam &
dedup_tool reads.sorted.bam > reads.sorted.dedup.bam &

wait
```

```
all: reads.sorted.dedup.bam

reads.qc.txt: reads.fastq
    qc_tool reads.fastq > reads.qc.txt

reads.trimmed.fastq: reads.fastq
    trim_tool reads.fastq > reads.trimmed.fastq

reads.sam: reads.trimmed.fastq reference.fa
    align_tool reference.fa reads.trimmed.fastq > reads.sam

reads.sorted.bam: reads.sam
    sort_tool reads.sam > reads.sorted.bam

reads.sorted.dedup.bam: reads.sorted.bam
    dedup_tool reads.sorted.bam > reads.sorted.dedup.bam


# Run entire pipeline with 4 allowed parallel jobs
make -j 4

# Run only a part of the pipeline
make reads.sorted.bam
```

Huge improvement! Can we make it even better? Maybe use Python?

```python
import subprocess

subprocess.run("qc_tool reads.fastq > reads.qc.txt", shell=True, check=True)
subprocess.run("trim_tool reads.fastq > reads.trimmed.fastq", shell=True, check=True)
subprocess.run("align_tool ref.fa reads.trimmed.fastq > reads.sam", shell=True, check=True)
subprocess.run("sort_tool reads.sam > reads.sorted.bam", shell=True, check=True)
subprocess.run("dedup_tool reads.sorted.bam > reads.sorted.dedup.bam", shell=True, check=True)
```

```python
import os
import subprocess

def run(cmd, outfile):
    if os.path.exists(outfile):
        print(f"{outfile} exists, skipping")
        return
    subprocess.run(cmd, shell=True, check=True)

run("qc_tool reads.fastq > reads.qc.txt", "reads.qc.txt")
run("trim_tool reads.fastq > reads.trimmed.fastq", "reads.trimmed.fastq")
run("align_tool ref.fa reads.trimmed.fastq > reads.sam", "reads.sam")
run("sort_tool reads.sam > reads.sorted.bam", "reads.sorted.bam")
run("dedup_tool reads.sorted.bam > reads.sorted.dedup.bam", "reads.sorted.dedup.bam")
```

```python
import subprocess
from multiprocessing import Process

def run(cmd):
    subprocess.run(cmd, shell=True, check=True)

if __name__ == "__main__":

    # Step 1: parallel QC + trimming
    p_qc = Process(target=run, args=("qc_tool reads.fastq > reads.qc.txt",))
    p_trim = Process(target=run, args=("trim_tool reads.fastq > reads.trimmed.fastq",))

    p_qc.start()
    p_trim.start()

    p_qc.join()
    p_trim.join()

    # Step 2: alignment
    run("align_tool ref.fa reads.trimmed.fastq > reads.sam")

    # Step 3: parallel sort + (hypothetical) indexing
    p_sort = Process(target=run, args=("sort_tool reads.sam > reads.sorted.bam",))

    p_sort.start()
    p_sort.join()

    # Step 4: dedup
    run("dedup_tool reads.sorted.bam > reads.sorted.dedup.bam")
```

Some problems cannot be solved by simple scripts

- Maximising the utilisation of available cores - **scheduling is an NP-hard problem so we need to find at least a good approximate solution!**
- What if the file exists but the analysis was not successfully completed?
- What about non-linear pipelines?
- Keeping track of tools' versions is complicated
- Each change in the script can break the workflow if the developer is not careful

Reusability

Workflow managers

Why reuse?

- *Ad-hoc* scripts do not scale well
- In production, we need to:
  - Save time spent on common analyses
  - Isolate tasks with Conda/Docker/Singularity/...
  - Make the analysis reproducible - **important when publishing the results!**
  - **Completely avoid construction of a pipeline if someone else has already created a better and more reliable solution**

Snakemake

- https://snakemake.github.io/
- **Rules are written in YAML-based domain-specific language (DSL)**
- Supports complex, non-linear pipelines with many dependencies
- Allows Conda, Docker, Singularity, HPC and cloud execution
- User can control resource allocation (CPU, MEM) for each rule
- **Large pipelines tend to get complicated and hard to maintain**

```
rule bwa_mem:
    input:
        reads=["reads/{sample}.1.fastq", "reads/{sample}.2.fastq"],
        idx=multiext("genome", ".amb", ".ann", ".bwt", ".pac", ".sa"),
    output:
        "mapped/{sample}.bam",
    log:
        "logs/bwa_mem/{sample}.log",
    params:
        extra=r"-R '@RG\tID:{sample}\tSM:{sample}'",
        sorting="none",  # Can be 'none', 'samtools' or 'picard'.
        sort_order="queryname",  # Can be 'queryname' or 'coordinate'.
        sort_extra="",  # Extra args for samtools/picard.
    threads: 8
    wrapper:
        "v3.9.0/bio/bwa/mem"
```

Snakemake wrappers

- https://snakemake-wrappers.readthedocs.io/en/stable/index.html
- Pre-built rules
- Standardise inputs, outputs and extra arguments
- **Reduces boilerplate and simplifies pipelines**

```
# Determine which pipe command to use for converting to bam or sorting.
if sort == "none":
    # Simply convert to bam using samtools view.
    pipe_cmd = "samtools view {samtools_opts}"

elif sort == "samtools":
    # Add name flag if needed.
    if sort_order == "queryname":
        sort_extra += " -n"

    # Sort alignments using samtools sort.
    pipe_cmd = "samtools sort {samtools_opts} {sort_extra} -T {tmpdir}"

elif sort == "fgbio":
    if sort_order == "queryname":
        sort_extra += " -s Queryname"
    pipe_cmd = "fgbio SortBam -i /dev/stdin -o {snakemake.output[0]} {sort_extra}"

elif sort == "picard":
    # Sort alignments using picard SortSam.
    pipe_cmd = "picard SortSam {java_opts} {sort_extra} --INPUT /dev/stdin --TMP_DI

else:
    raise ValueError(f"Unexpected value for params.sort ({sort})")

with tempfile.TemporaryDirectory() as tmpdir:
    shell(
        "(bwa mem"
        " -t {snakemake.threads}"
        " {extra}"
        " {index}"
        " {snakemake.input.reads}"
        " | " + pipe_cmd + ") {log}"
    )
```

Snakemake workflow catalog

- https://snakemake.github.io/snakemake-workflow-catalog/index.html
- **Curated and standardised workflows**
- However, the workflows are usually not well documented
- But if you prefer Snakemake, **you should start by searching if there is an existing workflow for your use case**

Snakemake workflow catalog / Top workflows by stars / snakemake-workflows/rna-seq-star-deseq2

snakemake-workflows/rna-seq-star-deseq2

license MIT | issues 2 open | stars 351 | watchers 12 | GitHub page

RNA-seq workflow using STAR and DESeq2

Overview

Latest release: v3.1.1, **Last update:** 2025-12-18

Linting: linting: passed, **Formatting:** formatting: passed

Topics: snakemake | sciworkflows | reproducibility | gene-expression-analysis | deseq2

Wrappers: bio/bwa/index | bio/fastp | bio/multiqc | bio/reference/ensembl-annotation | bio/reference/ensembl-sequence | bio/samtools/faidx | bio/sra-tools/fasterq-dump | bio/star/align | bio/star/index

Deployment

Step 1: Install Snakemake and Snakedeploy

On this page

Overview
Deployment
Step 1: Install Snakemake and Snakedeploy
Step 2: Deploy workflow
Step 3: Configure workflow
Step 4: Run workflow
Step 5: Generate report
Configuration
General configuration
`DESeq2` differential expression analysis configuration
Sample and unit setup
sample sheet
unit sheet
`.fastq` file source
strandedness of library preparation protocol
adapter trimming and read filtering
Lexogen 3′ QuantSeq adapter trimming
Linting and formatting
Linting results
Formatting results

Nextflow

- https://www.nextflow.io/
- **Written in Groovy - makes it harder to learn rule DSL than with Snakemake**
- Extremely scalable, production-ready, automatically generates reports
- Allows Docker, Singularity, Kubernetes, HPC and cloud execution

```
script:
def args = task.ext.args ?: ''
def args2 = task.ext.args2 ?: ''
def prefix = task.ext.prefix ?: "${meta.id}"
def samtools_command = sort_bam ? 'sort' : 'view'
def extension = args2.contains("--output-fmt sam")   ? "sam" :
                args2.contains("--output-fmt cram")  ? "cram":
                sort_bam && args2.contains("-O cram")? "cram":
                !sort_bam && args2.contains("-C")    ? "cram":
                "bam"
def reference = fasta && extension=="cram"  ? "--reference ${fasta}" : ""
if (!fasta && extension=="cram") error "Fasta reference is required for CRAM output"
"""
INDEX=`find -L ./ -name "*.amb" | sed 's/\\.amb\$//'`

bwa mem \\
    $args \\
    -t $task.cpus \\
    \$INDEX \\
    $reads \\
    | samtools $samtools_command $args2 ${reference} --threads $task.cpus -o ${prefix}.${extension} -

cat <<-END_VERSIONS > versions.yml
"${task.process}":
    bwa: \$(echo \$(bwa 2>&1) | sed 's/^.*Version: //; s/Contact:.*\$//')
    samtools: \$(echo \$(samtools --version 2>&1) | sed 's/^.*samtools //; s/Using.*\$//')
END_VERSIONS
"""
```

nf-core

- https://nf-co.re/
- **Curated best-practice pipelines for bioinformatics**
- Versioned, containerised, CI-tested
- Avoids “reinventing the wheel“
- **If you don’t know what to use for some type of analysis, you should start here**

nf-core/rnaseq

Edit

RNA sequencing analysis pipeline using STAR, RSEM, HISAT2 or Salmon with gene/isoform counts and extensive quality control.

rna rna-seq

Launch version 3.22.2

https://github.com/nf-core/rnaseq

Introduction | Usage | Parameters | Output | Results | Releases | 3.22.2

Introduction

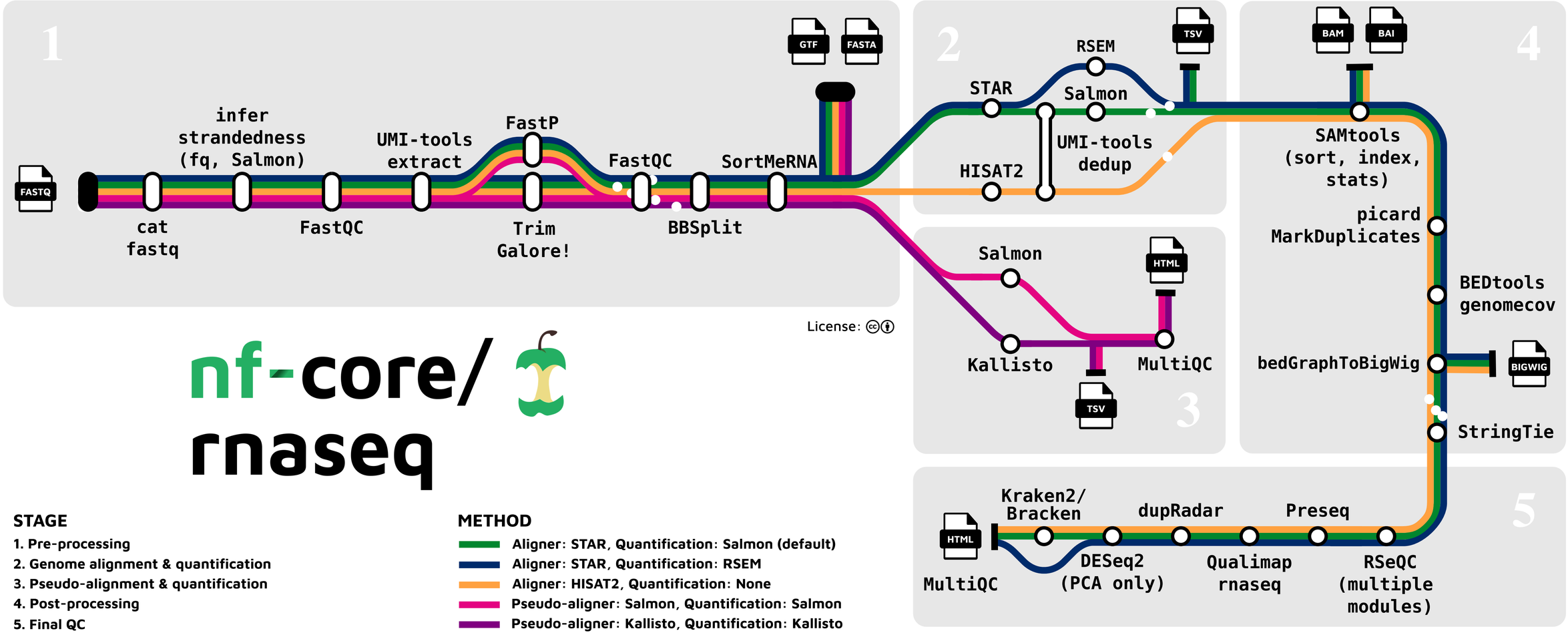
nf-core/rnaseq is a bioinformatics pipeline that can be used to analyse RNA sequencing data obtained from organisms with a reference genome and annotation. It takes a samplesheet with FASTQ files or pre-aligned BAM files as input, performs quality control (QC), trimming and (pseudo-)alignment, and produces a gene expression matrix and extensive QC report.

run with

```
nf-core pipelines launch nf
```

nf-core | Nextflow | Seqera Platform

video introduction



License: CC BY

nf-core/rnaseq

STAGE

1. Pre-processing
2. Genome alignment & quantification
3. Pseudo-alignment & quantification
4. Post-processing
5. Final QC

METHOD

- Aligner: STAR, Quantification: Salmon (default)
- Aligner: STAR, Quantification: RSEM
- Aligner: HISAT2, Quantification: None
- Pseudo-aligner: Salmon, Quantification: Salmon
- Pseudo-aligner: Kallisto, Quantification: Kallisto

Cloud Genomics (AWS / Azure)

- https://aws.amazon.com/health/genomics/
- https://www.microsoft.com/en-us/research/project/microsoft-genomics/
- **User-friendly implementations of workflow managers in cloud**
- **Useful when you don't have local compute cluster**
- Also useful when you have too much money - very expensive
- **Security concerns**

Illumina DRAGEN

- https://www.illumina.com/products/by-type/informatics-products/dragen-secondary-analysis.html
- **Hardware-accelerated genomics, reduces processing time to minutes**
- Newer Illumina sequencers have DRAGEN built-in
- **Limited flexibility and customisation options**
- Very expensive x2
- DRAGEN is also available through AWS / Azure Genomics —> price++

```
dragen \
  --fastq-file1 sample_R1.fastq.gz --fastq-file2 sample_R2.fastq.gz \
  --enable-trimming true --trim-min-length 35 \
  --trim-adapter-read1 AGATCGGAAGAGCACACGTCTGAACTCCAGTCA \
  --trim-adapter-read2 AGATCGGAAGAGCGTCGTGTAGGGAAAGAGTGT \
  --enable-fastq-qc true --fastq-qc-output-dir fastq_qc \
  --ref-dir /opt/dragen/ref/GRCh38 --enable-map-align true \
  --enable-sort true --enable-duplicate-marking true \
  --enable-bam-indexing true --enable-map-align-output true \
  --aligner-mode bwa-mem --aligner-max-seed 2 \
  --aligner-score-mode random --duplicate-marking-method hash \
  --enable-duplicate-marking true --output-directory output \
  --output-file-prefix sample --enable-bam-qc true \
  --bam-qc-output-dir bam_qc --enable-variant-caller true \
  --vc-target-bed targets.bed --vc-emit-ref-confidence GVCF \
  --vc-enable-vcf-output true --vc-enable-gvcf-output true \
  --vc-genotype-gvcfs false --vc-enable-joint-genotyping false \
  --vc-sample-name sample --enable-sv true \
  --sv-call-regions sv_regions.bed --sv-exome false \
  --enable-cnv true --cnv-target-bed targets.bed \
  --cnv-enable-self-normalization true \
  --enable-msi true --msi-coverage-threshold 20 \
  --intermediate-results-dir intermediate \
  --log-dir logs --log-level info
```

Questions?